



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА №3

«Разработка приложения на языке C# с базой данных и
пользовательским интерфейсом»

**ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки
информации и управления»**

Выполнил: студент гр. ИУ5-22Б

(Подпись) (Краснова А. А.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк О. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

1. Цель работы

Разработать приложение на языке C# с использованием Entity Framework Core для доступа к базе данных SQLite и графическим интерфейсом Windows Forms, реализующее CRUD операции над справочной и основной таблицами, а также формирование отчетов с помощью LINQ to Entities.

2. Требования

В соответствии с методическими указаниями, разрабатываемое приложение должно соответствовать следующим требованиям:

- Язык программирования: C#
- База данных: SQLite
- Доступ к данным: Entity Framework Core (Code First)
- Запросы к данным: LINQ to Entities
- Интерфейс пользователя: Windows Forms

3. Вариант задания

Вариант 17 по списку группы.

- Справочная таблица (master): Издательства (Publishers)
- Основная таблица (detail): Журналы (Journals)
- Числовое поле: Тираж (circulation_k) в тысячах экземпляров

4. Часть 1. Модель данных и DbContext

4.1 Классы-модели

Для Entity Framework Core были созданы два класса-модели, адаптированные из домашнего задания №2.

Класс Publisher (справочная таблица, сторона "один"):

- Свойство Id - первичный ключ
- Свойство Name - название издательства
- Навигационное свойство Journals - коллекция журналов данного издательства, реализующее связь один-ко-многим

```
}  
using System.ComponentModel.DataAnnotations;  
  
namespace Homework3.Models  
{  
    public class Publisher  
    {
```

```

[Key]
public int Id { get; set; }

public string Name { get; set; } = string.Empty;

public ICollection<Journal> Journals { get; set; } = new List<Journal>();

public Publisher() { }

public Publisher(int id, string name)
{
    Id = id;
    Name = name;
}

public override string ToString() => $"{Name}";
}
}

```

Класс Journal (основная таблица, сторона "много"):

- Свойство Id - первичный ключ
- Свойство PublisherId - внешний ключ на издательство
- Свойство Name - название журнала
- Свойство CirculationK - тираж в тысячах экземпляров
- Навигационное свойство Publisher - ссылка на издательство
- Валидация: значение тиража не может быть отрицательным

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Homework3.Models
{
    /// <summary>Журнал (основная таблица)</summary>
    public class Journal
    {
        [Key]
        public int Id { get; set; }

        [ForeignKey("Publisher")]
        public int PublisherId { get; set; }

        public string Name { get; set; } = string.Empty;

        private int _circulationK;

        /// <summary>Тираж (тыс. экз.) - не может быть отрицательным</summary>
        public int CirculationK
        {
            get => _circulationK;
            set
            {
                if (value < 0)
                {
                    throw new ArgumentException("Тираж не может быть отрицательным!");
                }
                _circulationK = value;
            }
        }

        /// <summary>Навигационное свойство: издательство журнала</summary>
        public virtual Publisher? Publisher { get; set; }

        public Journal() { }
    }
}

```

```

        public Journal(int id, int publisherId, string name, int circulationK)
        {
            Id = id;
            PublisherId = publisherId;
            Name = name;
            CirculationK = circulationK;
        }

        public override string ToString() => $"{Name} (тираж: {CirculationK} тыс. экз.)";
    }
}

```

Все публичные классы и их публичные члены снабжены XML-комментариями.

4.2 Контекст базы данных

Создан класс AppDbContext, наследующий от DbContext:

- Содержит DbSet Publisher и DbSet Journal
- В методе OnConfiguring настроено подключение к SQLite с файлом базы данных app.db
- В методе OnModelCreating настроена связь один-ко-многим с запретом каскадного удаления

4.3 Создание базы данных и начальные данные

При первом запуске приложение автоматически создает базу данных с помощью метода context.Database.EnsureCreated.

Начальные данные загружаются из CSV-файлов:

- publishers.csv - содержит 4 записи издательств
- journals.csv - содержит 12 записей журналов

Если CSV-файлы отсутствуют, используются данные по умолчанию.

```

using Microsoft.EntityFrameworkCore;

namespace Homework3.Models
{
    public class AppDbContext : DbContext
    {
        public DbSet<Publisher> Publishers { get; set; }
        public DbSet<Journal> Journals { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            string dbPath = Path.Combine(AppContext.BaseDirectory, "app.db");
            optionsBuilder.UseSqlite($"Data Source={dbPath}");
        }
    }
}

```

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Journal>()
        .HasOne(j => j.Publisher)
        .WithMany(p => p.Journals)
        .HasForeignKey(j => j.PublisherId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Publisher>()
        .HasIndex(p => p.Name)
        .IsUnique();

    base.OnModelCreating(modelBuilder);
}
}
}

```

5. Форма для справочной таблицы (PublishersForm)

Реализована форма для работы с издательствами, включающая следующие операции:

- Просмотр

Отображается список всех издательств, отсортированных по названию. Данные выводятся в DataGridView с настройкой заголовков и ширины колонок.

- Добавление

Пользователь вводит название нового издательства в текстовое поле. При нажатии кнопки "Добавить" запись сохраняется в базу данных через Entity Framework Core. Выполняется проверка на пустое название и на уникальность.

- Редактирование

Пользователь выбирает строку в таблице, после чего название загружается в текстовое поле. После изменения названия и нажатия кнопки "Редактировать" изменения сохраняются в базе данных.

- Удаление

Пользователь выбирает издательство и нажимает кнопку "Удалить". Выводится диалог подтверждения. Перед удалением выполняется проверка наличия связанных журналов. Если у издательства есть хотя бы один журнал, удаление запрещается и выводится соответствующее сообщение.

```

private void btnAdd_Click(object sender, EventArgs e)
{
    string name = txtName.Text.Trim();

    if (string.IsNullOrEmpty(name))

```

```

    {
        MessageBox.Show("Введите название!", "Ошибка", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        return;
    }

    try
    {
        using var context = new AppDbContext();
        context.Publishers.Add(new Publisher { Name = name });
        context.SaveChanges();
        MessageBox.Show("Добавлено!", "Успех", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
        txtName.Clear();
        LoadData();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

```

Все операции с базой данных выполняются через LINQ to Entities. Прямые SQL-запросы не используются. При ошибках ввода выводится понятное сообщение пользователю.

6. Форма для основной таблицы (JournalsForm)

Реализована форма для работы с журналами, включающая следующие операции:

- Просмотр

Отображается список всех журналов с отображением названия издательства (не просто идентификатора). Для этого используется метод Include для подгрузки связанных данных.

```

using var context = new AppDbContext();
context.Journals.Add(new Journal(0, publisherId, name, circulation));
context.SaveChanges();

```

- Добавление

Пользователь вводит название журнала, выбирает издательство из выпадающего списка (загруженного из справочника) и указывает тираж. Выпадающий список формируется LINQ-запросом к справочнику издательств.

```

using var context = new AppDbContext();
var journal = context.Journals.Find(_selectedJournal.Id);
if (journal != null)
{
    journal.Name = name;
    journal.PublisherId = publisherId;
    journal.CirculationK = circulation;
    context.SaveChanges();
}

```

- Редактирование

При выборе журнала из таблицы его данные загружаются в поля формы. После изменения пользователь сохраняет изменения. Издательство также выбирается из выпадающего списка.

- Удаление

Пользователь выбирает журнал и нажимает кнопку "Удалить". Выводится диалог подтверждения, после чего запись удаляется из базы данных.

- Валидация

Реализована проверка числового поля тираж: значение не может быть отрицательным. При некорректном вводе выводится сообщение об ошибке.

```
private void btnEdit_Click(object sender, EventArgs e)
{
    if (_selectedJournal == null)
    {
        MessageBox.Show("Выберите журнал!", "Ошибка", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        return;
    }

    string name = txtName.Text.Trim();

    if (string.IsNullOrEmpty(name))
    {
        MessageBox.Show("Введите название журнала!", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (cmbPublisher.SelectedValue == null)
    {
        MessageBox.Show("Выберите издательство!", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int publisherId = (int)cmbPublisher.SelectedValue;
    int circulation = (int)numCirculation.Value;

    if (circulation < 0)
    {
        MessageBox.Show("Тираж не может быть отрицательным!", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    try
    {
        using var context = new AppDbContext();
        var journal = context.Journals.Find(_selectedJournal.Id);
        if (journal != null)
        {
            journal.Name = name;
            journal.PublisherId = publisherId;
            journal.CirculationK = circulation;
            context.SaveChanges();
        }
    }
}
```

```

        MessageBox.Show("Журнал обновлён!", "Успех", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
        ClearSelection();
        LoadData();
    }
}
catch (ArgumentException ex)
{
    MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
}
}

private void btnDelete_Click(object sender, EventArgs e)
{
    if (_selectedJournal == null)
    {
        MessageBox.Show("Выберите журнал!", "Ошибка", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        return;
    }

    var result = MessageBox.Show($"Удалить журнал \"{_selectedJournal.Name}\"?",
    "Подтверждение", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        try
        {
            using var context = new AppDbContext();
            var journal = context.Journals.Find(_selectedJournal.Id);
            if (journal != null)
            {
                context.Journals.Remove(journal);
                context.SaveChanges();
                MessageBox.Show("Журнал удалён!", "Успех", MessageBoxButtons.OK,
                MessageBoxIcon.Information);
                ClearSelection();
                LoadData();
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
    }
}
}

```

Все запросы к базе данных выполняются через LINQ to Entities с использованием Include для подгрузки связанных данных.

7. Отчеты

Реализована отдельная форма для вывода отчета, содержащая три раздела, сформированных средствами LINQ to Entities.

Раздел 1. Полный список. Вывод всех записей основной таблицы с названием категории из справочника, отсортированных по названию журнала.

Раздел 2. Количество записей по категориям. Группировка записей по издательствам с использованием агрегатной функции Count.

Раздел 3. Среднее значение числового поля по категориям. Группировка по издательствам с использованием агрегатной функции Average, сортировка по убыванию среднего значения. Значение округлено до двух знаков после запятой.

Все три раздела отображаются в одной форме с использованием элементов управления TabControl. Результаты выводятся в виде таблиц DataGridView.

```
private void LoadReports()
{
    using var context = new AppDbContext();
    LoadReport1(context);
    LoadReport2(context);
    LoadReport3(context);
}

private void LoadReport1(AppDbContext context)
{
    var report1 = context.Journals
        .Include(j => j.Publisher)
        .OrderBy(j => j.Name)
        .Select(j => new
        {
            Название = j.Name,
            Издательство = j.Publisher != null ? j.Publisher.Name : "Не указано",
            Тираж_тыс_экз = j.CirculationK
        })
        .ToList();

    dgvReport1.DataSource = report1;
    lblReport1Count.Text = $"Всего записей: {report1.Count}";
}

private void LoadReport2(AppDbContext context)
{
    var report2 = context.Journals
        .Include(j => j.Publisher)
        .GroupBy(j => j.Publisher != null ? j.Publisher.Name : "Не указано")
        .Select(g => new
        {
            Издательство = g.Key,
            Количество = g.Count()
        })
        .OrderBy(r => r.Издательство)
        .ToList();

    dgvReport2.DataSource = report2;
    int total = report2.Sum(r => r.Количество);
    lblReport2Count.Text = $"Всего журналов: {total}";
}

private void LoadReport3(AppDbContext context)
{
    var report3 = context.Journals
        .Include(j => j.Publisher)
        .GroupBy(j => j.Publisher != null ? j.Publisher.Name : "Не указано")
        .Select(g => new
        {
            Издательство = g.Key,
            Средний_тираж = g.Average(j => j.CirculationK)
        })
        .OrderByDescending(r => r.Средний_тираж)
        .ToList();
}
```

```
}    dgvReport3.DataSource = report3;  
}
```

8. Навигация

Реализовано главное окно приложения с тремя кнопками для перехода к соответствующим разделам:

- Кнопка "Издательства" открывает форму справочной таблицы
- Кнопка "Журналы" открывает форму основной таблицы
- Кнопка "Отчеты" открывает форму с отчетами
- Кнопка "Выход" завершает работу приложения

```
public MainForm()  
{  
    InitializeComponent();  
}  
  
private void btnPublishers_Click(object sender, EventArgs e)  
{  
    var form = new PublishersForm();  
    form.ShowDialog();  
}  
  
private void btnJournals_Click(object sender, EventArgs e)  
{  
    var form = new JournalsForm();  
    form.ShowDialog();  
}  
  
private void btnReport_Click(object sender, EventArgs e)  
{  
    var form = new ReportForm();  
    form.ShowDialog();  
}  
  
private void btnExit_Click(object sender, EventArgs e)  
{  
    Application.Exit();  
}
```

9. Структура проекта

Проект имеет следующую структуру папок и файлов:

Папка Models содержит:

- Publisher.cs - класс издательства
- Journal.cs - класс журнала
- AppDbContext.cs - контекст базы данных
- DataSeeder.cs - загрузка начальных данных из CSV

Папка Forms содержит:

- MainForm.cs - главная форма с навигацией
- PublishersForm.cs - форма для работы с издательствами
- JournalsForm.cs - форма для работы с журналами
- ReportForm.cs - форма для вывода отчетов

Папка Data содержит:

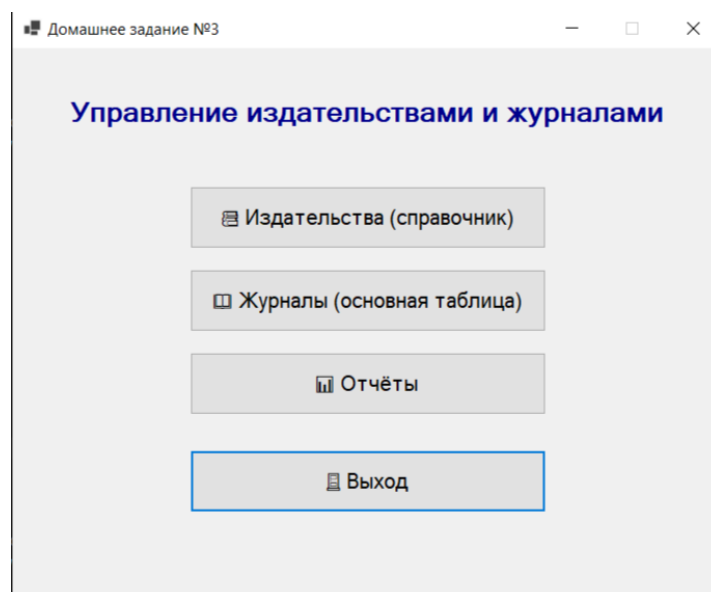
- publishers.csv - начальные данные издательств
- journals.csv - начальные данные журналов

Файл Program.cs является точкой входа в приложение. В нем выполняется инициализация базы данных и запуск главной формы.

```
Homework3/
├── Program.cs           # Точка входа
├── Models/
│   ├── Publisher.cs    # Модель издательства
│   ├── Journal.cs      # Модель журнала
│   ├── AppDbContext.cs # Контекст БД
│   └── DataSeeder.cs   # Загрузка начальных данных из CSV
├── Forms/
│   ├── MainForm.cs     # Главная форма (навигация)
│   ├── PublishersForm.cs # CRUD для издательств
│   ├── JournalsForm.cs  # CRUD для журналов
│   └── ReportForm.cs    # Форма отчётов
└── Data/
    ├── publishers.csv   # Начальные данные издательств
    └── journals.csv
```

10. Скриншоты, демонстрирующие работу программы

10.1 Главное окно



10.2 Раздел управления издательствами

Управление издательствами

	Id	Name
▶	4	Domino
	2	Education
	1	Science
	3	Sport

Всего издательств: 4

Название:
Domino

+ Добавить

✎ Редактировать

🗑 Удалить

🔄 Обновить

✕ Закрыть

10.3 Раздел управление журналами

Управление журналами

	Id	Name	PublisherName	CirculationK
▶	110	About sport	Education	90
	108	All for mum	Domino	150
	105	Earth	Sport	120
	107	For car	Domino	200
	109	Helth	Science	30
	114	Mumu	Education	80
	102	Nature	Science	20
	111	Phisics everywhere	Sport	40
	101	Quantum	Science	45
	103	Technic	Education	55
	112	Time to live	Domino	110
	106	Yuang	Sport	60

Всего журналов: 12

Название
About sport

Издательство:
Education

Тираж (тыс. экз.):
90

Добавить

Редактировать

Удалить

Обновить

Закрыть

10.4 Раздел отчетов

Отчеты

Отчеты

1. Полный список журналов 2. Количество по издательствам 3. Средний тираж по издательствам

Название	Издательство	Тираж_тыс_экз
About sport	Education	90
All for mum	Domino	150
Earth	Sport	120
For car	Domino	200
Helth	Science	30
Mumu	Education	80
Nature	Science	20
Physics everywhere	Sport	40
Quantum	Science	45
Technic	Education	55
Time to live	Domino	110
Yuang	Sport	60

Всего записей: 12

Обновить Закрыть

11. Заключение

В ходе выполнения домашней работы было разработано приложение на языке C# .

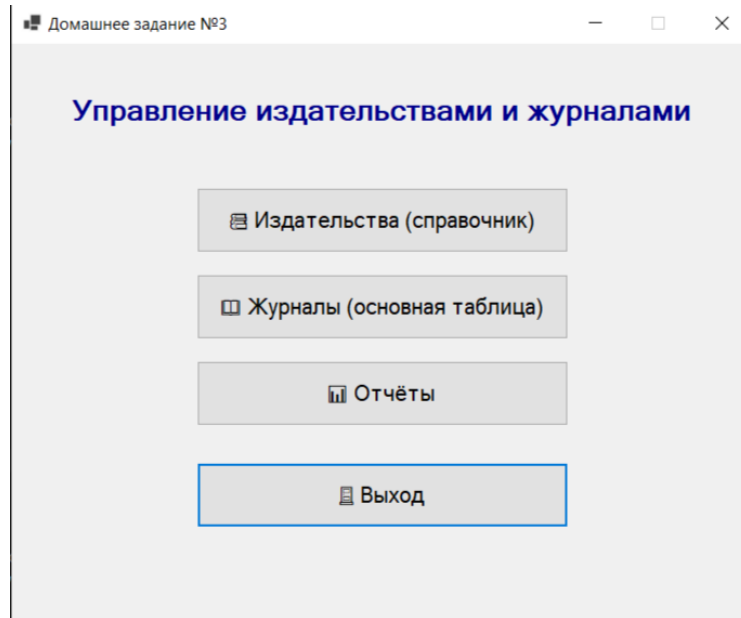
Entity Framework Core обеспечил удобную работу с базой данных SQLite без написания прямых SQL-запросов, что соответствует требованиям методических указаний.

LINQ to Entities позволил формировать сложные запросы для отчетов, включая группировку и агрегатные функции Count и Average.

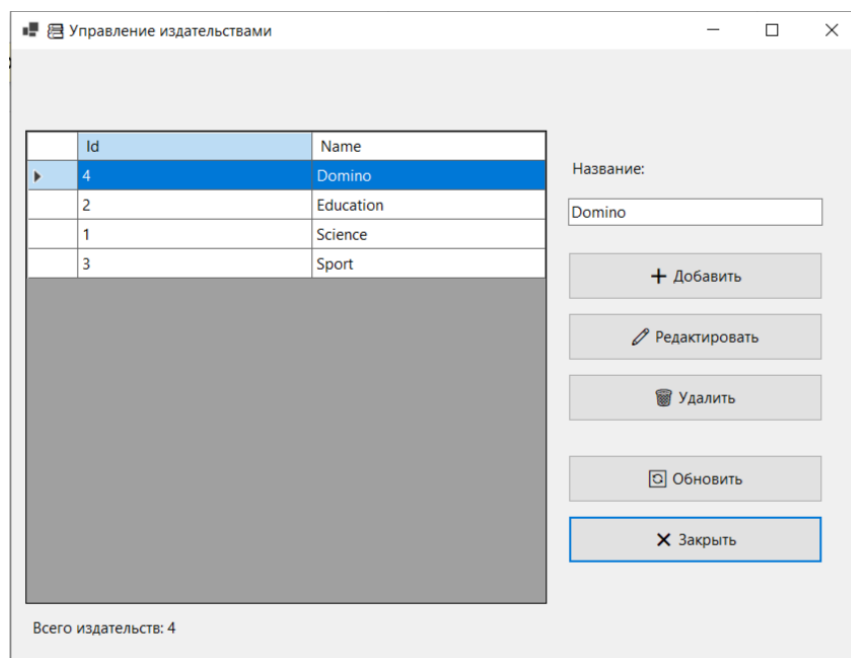
Графический интерфейс на Windows Forms предоставил интуитивно понятный способ взаимодействия с пользователем, включающий формы для выполнения CRUD операций над справочной и основной таблицами, а также форму для просмотра аналитических отчетов.

9. Скриншоты работы приложения

9.1 Главное окно



9.2 Форма управления издательствами



9.3 Форма управления журналами

Управление журналами

	Id	Name	PublisherName	CirculationK
▶	110	About sport	Education	90
	108	All for mum	Domino	150
	105	Earth	Sport	120
	107	For car	Domino	200
	109	Helth	Science	30
	114	Mumu	Education	80
	102	Nature	Science	20
	111	Phisics everywhere	Sport	40
	101	Quantum	Science	45
	103	Technic	Education	55
	112	Time to live	Domino	110
	106	Yuang	Sport	60

Всего журналов: 12

Название

About sport

Издательство:

Education

Тираж (тыс. экз.):

90

Добавить

Редактировать

Удалить

Обновить

Заккрыть

9.4 Форма отчётов

Отчеты

Отчеты

1. Полный список журналов

2. Количество по издательствам

3. Средний тираж по издательствам

	Название	Издательство	Тираж_тыс_экз
▶	About sport	Education	90
	All for mum	Domino	150
	Earth	Sport	120
	For car	Domino	200
	Helth	Science	30
	Mumu	Education	80
	Nature	Science	20
	Phisics everywhere	Sport	40
	Quantum	Science	45
	Technic	Education	55
	Time to live	Domino	110
	Yuang	Sport	60

Всего записей: 12

Обновить

Заккрыть